

Chemicals that Drive Aroma Ratings

We want to remove the effect of the subjects from our response, this will give us better results for chemicals that drive aroma scores. We will residualize the response by fitting a model for Aroma ~ Subject variables, then find the residuals, we will then use the residuals as our response and implement 3 things to screen and test screening ElasticNet, SIS and PLS. We will also use ML models like XGBoost to find the important variables but I'd be very concerned about using them because of the size of our dataset.

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.0      v readr      2.2.0
## v forcats    1.0.1      v stringr    1.6.0
## v ggplot2    4.0.2      v tibble     3.3.1
## v lubridate  1.9.5      v tidyr      1.3.2
## v purrr      1.2.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(lme4)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

```
library(lmerTest)
```

```
##
## Attaching package: 'lmerTest'
##
## The following object is masked from 'package:lme4':
##
##     lmer
##
## The following object is masked from 'package:stats':
##
##     step
```

```
library(glmnet)
```

```
## Loaded glmnet 4.1-10
```

```
subject_df <- read.csv("Subject_Level_Dairy_Cheese_latent_factors.csv")
df_chem <- read.csv("Cheese_Chemical_Data.csv")
rating_df <- read.csv("Subject_wise_Cheese_Ratings.csv")
```

Variables to consider for subject level data: "Frequency_of_PB_cheese", "Diet", "Age", "Cheese_MR1", "Cheese_MR2".

Data Prep:

Chemical Data:

```

chem_cols <- colnames(df_chem)[4:54]

chem_screen_df <- df_chem %>%
  select(all_of(c("PrimaryID", chem_cols)))

#Log-transforming the chemical data
chem_screen_df[, 2:52] <- log(chem_screen_df[, 2:52])

X_chem_scaled <- chem_screen_df

X_chem_scaled[, chem_cols] = chem_screen_df %>%
  select(all_of(chem_cols)) %>%
  scale()
head(X_chem_scaled)

## PrimaryID Feature1_N Feature2_N Feature3_N Feature4_N Feature5_N Feature6_N
## 1 B1 0.6893905 0.80269129 0.03797742 -0.9428820 -0.3226079 -1.3905872
## 2 B2 0.7037186 1.14438823 0.06378428 -0.8485840 -0.2616387 -1.2666397
## 3 B3 0.5267816 0.75183290 -0.06663042 -0.9924851 -0.3622921 -1.1639939
## 4 B4 0.6287979 1.05433407 -0.12140677 -1.0260518 -0.3445874 -1.3398077
## 5 H1 -0.6181839 -0.09092833 -1.78291657 0.7018053 -0.1484829 -0.1630372
## 6 H2 -0.6114874 -0.04430066 -1.86296518 0.7129243 -0.1078311 -0.2101553
## Feature7_N Feature8_N Feature9_N Feature10_N Feature11_N Feature12_N
## 1 1.0356675 2.0116794 -1.5913646 0.29178374 -0.2032154 0.45869421
## 2 1.3860902 2.2401856 -1.5476263 0.74723383 -0.2094630 -0.06928947
## 3 0.8351870 1.8508413 -1.6651711 -0.02270404 -0.1817126 -0.01535037
## 4 0.8280290 1.7353556 -1.6866915 -0.26486385 -0.3227123 0.32896942
## 5 0.6214222 -0.6252084 -0.4531404 -0.94185749 1.4696792 0.43874889
## 6 0.6459863 -1.0063347 -0.5482500 -0.60920293 1.2604695 -0.38795573
## Feature13_N Feature14_N Feature15_N Feature16_N Feature17_N Feature18_N
## 1 0.23326563 0.879356663 0.2694543 -0.4189349 -1.6129851 0.8133187
## 2 0.13893473 1.197148490 0.4138536 -0.4018761 -1.5388113 0.9313632
## 3 0.11372875 1.243697107 0.2233222 -0.4500950 -1.8951940 0.8949527
## 4 -0.04243828 0.934318412 0.1557937 -0.5361504 -1.6403834 0.8027698
## 5 -0.57702799 0.003298322 -1.0560536 -1.1353937 -0.2901536 0.5770288
## 6 0.30350677 0.092183826 -1.0823828 -1.1030463 -0.1729445 0.6517487
## Feature19_N Feature20_N Feature21_N Feature22_N Feature23_N Feature24_N
## 1 1.3918851 1.6746112 1.0566895 1.1718376 2.319928 -0.3813318
## 2 1.4296739 1.6325714 1.1028601 1.0572722 2.385574 -0.2652761
## 3 1.3901518 1.6744774 1.0588308 1.1524082 2.386396 -0.3279343
## 4 1.3473485 1.4828805 1.0024851 -0.3030816 2.301727 -0.4717660
## 5 -0.4402094 0.2733362 -0.3455140 -1.8776812 -1.134931 1.2727995
## 6 -0.4878531 0.7053871 -0.3130375 -0.7562652 -1.228548 1.2598021
## Feature25_N Feature26_N Feature27_N Feature28_N Feature29_N Feature30_N
## 1 -1.0413366 0.2609714 -0.43796473 -0.6859650 0.7118181 -1.0466293
## 2 -1.3939414 0.3595591 -0.34200411 0.5279102 0.1556334 -1.0184635
## 3 -1.2538847 0.6303465 -0.05625364 1.4930452 0.6518560 -0.8715829
## 4 -1.2269178 0.5877708 -0.30746223 0.8301318 0.4234733 -1.0222872
## 5 0.9455496 -1.6842955 0.61885739 -0.9960790 -1.2260046 1.1377656
## 6 0.9514030 -0.7444176 0.81263447 -1.2461714 -1.6203983 1.1360960
## Feature31_N Feature32_N Feature33_N Feature34_N Feature35_N Feature36_N
## 1 -1.4082637 0.75579138 2.258534 -1.8361358 -0.3024641 -0.6846651

```

```
## 2  0.6888947  0.00426548    2.289554 -2.0945953 -0.6499123 -0.8604411
## 3  1.2405390  0.78793994    2.321344 -1.9958819 -0.7215282 -0.3353748
## 4  0.4497552 -0.52407406    2.142213 -2.2871001 -0.7293226 -0.7539560
## 5 -1.0085167 -2.35513094    1.178978 -0.3284173 -0.5823916 -0.2003477
## 6 -0.6621753  0.97185359    1.165133 -0.3147080 -0.8415178 -0.3874469
##   Feature37_N Feature38_N Feature39_N Feature40_N Feature41_N Feature42_N
## 1 -0.009480365 -0.4217032  0.4651913  0.4294250  2.0921630 -0.8744897
## 2 -0.024205702 -0.4440054  0.4489720  0.4094146  2.1083939 -0.9401084
## 3  0.005092773 -0.5179867  0.3633479  0.3074336  1.8872290 -0.9322936
## 4  0.032896806 -0.4687601  0.3906217  0.3464810  1.9998064 -1.0246659
## 5 -1.117079519 -1.3273683 -1.4964124 -1.5990882 -1.1356817 -1.3181020
## 6 -0.590466917 -0.8471628 -1.5242098 -1.3378364 -0.9079525 -1.1248715
##   Feature43_N Feature44_N Feature45_N Feature46_N Feature47_N Feature48_N
## 1  0.9063967  1.448780  0.4787488 -1.4171812 -0.84670225  1.559187
## 2  1.2502925  1.371652  0.4242311 -1.2611809 -0.21391454  1.491215
## 3  0.5111410  1.301197  0.3503730 -1.1978590 -1.00458479  1.442893
## 4  0.6875625  1.382232  0.3982833 -1.5409253 -0.77068518  1.478009
## 5 -0.8697366 -1.542325 -0.3160379  0.1533545 -0.33707788 -1.351666
## 6 -0.7898184 -1.301797 -0.2557081  0.1871684 -0.08267857 -1.289198
##   Feature49_N Feature50_N Feature51_N
## 1 -2.3714597 -0.5111920 -2.3694059
## 2 -2.6120254 -0.6249438 -2.7268350
## 3 -2.8003392 -0.7085092 -3.1459671
## 4 -2.7050015 -0.6673633 -2.7290436
## 5 -0.7694968 -0.3327001 -0.5022457
## 6 -0.7205130 -0.3001561 -0.3666590
```

Ratings and Subject Data:

```
subject_vars <- c("Frequency_of_PB_cheese", "Diet",
                  "Age", "Cheese_MR1", "Cheese_MR2")

analysis_df <- rating_df %>%
  left_join(subject_df, by = "Unique_Panelist_ID")

needed_cols <- c("Unique_Panelist_ID", "PrimaryID", "Aroma_A", subject_vars)

analysis_df <- analysis_df %>%
  select(all_of(needed_cols))

analysis_df$Diet <- factor(analysis_df$Diet)

analysis_df$Age <- factor(analysis_df$Age, ordered = T)
analysis_df$Unique_Panelist_ID <- factor(analysis_df$Unique_Panelist_ID)
```

Mixed effects model with a random intercept for subject:

```
m_subject <- lmer(data = analysis_df,
                  formula = Aroma_A ~ (1|Unique_Panelist_ID)+
                    Frequency_of_PB_cheese + Age + Diet + Cheese_MR1 + Cheese_MR2,
                  REML = TRUE)

#Residualizing the response
analysis_df$resid_subject <- resid(m_subject)

summary(m_subject)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: Aroma_A ~ (1 | Unique_Panelist_ID) + Frequency_of_PB_cheese +
##      Age + Diet + Cheese_MR1 + Cheese_MR2
##      Data: analysis_df
##
## REML criterion at convergence: 29546.3
##
## Scaled residuals:
##      Min       1Q   Median       3Q      Max
## -2.6017 -0.7425  0.1072  0.7717  2.3715
##
## Random effects:
##      Groups                Name             Variance Std.Dev.
## Unique_Panelist_ID (Intercept) 0.842         0.9176
## Residual                      3.749         1.9362
## Number of obs: 7020, groups: Unique_Panelist_ID, 135
##
## Fixed effects:
##              Estimate Std. Error      df t value Pr(>|t|)
## (Intercept)    4.976e+00  1.869e-01  1.220e+02  26.629  <2e-16 ***
## Frequency_of_PB_cheese  7.647e-02  5.801e-02  1.220e+02   1.318    0.190
## Age.L          -9.889e-02  3.432e-01  1.220e+02  -0.288    0.774
## Age.Q          -3.925e-03  3.168e-01  1.220e+02  -0.012    0.990
## Age.C          -1.174e-02  2.549e-01  1.220e+02  -0.046    0.963
## Age^4           1.071e-01  2.074e-01  1.220e+02   0.516    0.607
## Age^5           4.400e-06  1.822e-01  1.220e+02   0.000    1.000
## Diet2          -3.596e-01  2.984e-01  1.220e+02  -1.205    0.230
## Diet3           2.359e-01  7.040e-01  1.220e+02   0.335    0.738
## Diet4          -2.586e-01  3.636e-01  1.220e+02  -0.711    0.478
## Diet5          -7.260e-01  5.288e-01  1.220e+02  -1.373    0.172
## Cheese_MR1       8.144e-02  9.352e-02  1.220e+02   0.871    0.386
## Cheese_MR2      -2.499e-02  9.693e-02  1.220e+02  -0.258    0.797
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation matrix not shown by default, as p = 13 > 12.
## Use print(x, correlation=TRUE) or
##      vcov(x)          if you need it
```

Cheese level data for response:

```
cheese_target_df <- analysis_df %>%
  group_by(PrimaryID) %>%
  summarise(
    mean_rating = mean(Aroma_A),
    mean_resid_subject = mean(resid_subject),
    n_ratings = n(),
    .groups = "drop"
  )
names(cheese_target_df)[1] <- "PrimaryID"

write.csv(cheese_target_df,
```

```
"cheese_level_residualized_target.csv", row.names = FALSE)
```

Merging residualized response with cheese level data (chemicals):

```
screen_df <- cheese_target_df %>%
  left_join(X_chem_scaled, by = "PrimaryID")
```

```
screen_df[!complete.cases(screen_df),]
```

```
## # A tibble: 0 x 55
## # i 55 variables: PrimaryID <chr>, mean_rating <dbl>, mean_resid_subject <dbl>,
## #   n_ratings <int>, Feature1_N <dbl>, Feature2_N <dbl>, Feature3_N <dbl>,
## #   Feature4_N <dbl>, Feature5_N <dbl>, Feature6_N <dbl>, Feature7_N <dbl>,
## #   Feature8_N <dbl>, Feature9_N <dbl>, Feature10_N <dbl>, Feature11_N <dbl>,
## #   Feature12_N <dbl>, Feature13_N <dbl>, Feature14_N <dbl>, Feature15_N <dbl>,
## #   Feature16_N <dbl>, Feature17_N <dbl>, Feature18_N <dbl>, Feature19_N <dbl>,
## #   Feature20_N <dbl>, Feature21_N <dbl>, Feature22_N <dbl>, ...
```

```
rbind(head(screen_df), tail(screen_df))
```

```
## # A tibble: 12 x 55
##   PrimaryID mean_rating mean_resid_subject n_ratings Feature1_N Feature2_N
##   <chr>         <dbl>         <dbl>         <int>         <dbl>         <dbl>
## 1 A1           5.76           0.623           135           1.77          -0.681
## 2 A2           5.76           0.623           135           1.72          -0.878
## 3 A3           5.76           0.623           135           1.68          -1.09
## 4 A4           5.76           0.623           135           1.58          -0.929
## 5 B1           4.64          -0.488           135           0.689           0.803
## 6 B2           4.64          -0.488           135           0.704           1.14
## 7 M3           4.41          -0.717           135          -0.387          -2.61
## 8 M4           4.41          -0.717           135          -0.624          -2.52
## 9 N1           6.05           0.920           135          -0.567          -0.440
## 10 N2          6.05           0.920           135          -0.706          -0.443
## 11 N3          6.05           0.920           135          -0.605          -0.499
## 12 N4          6.05           0.920           135          -0.621          -0.625
## # i 49 more variables: Feature3_N <dbl>, Feature4_N <dbl>, Feature5_N <dbl>,
## #   Feature6_N <dbl>, Feature7_N <dbl>, Feature8_N <dbl>, Feature9_N <dbl>,
## #   Feature10_N <dbl>, Feature11_N <dbl>, Feature12_N <dbl>, Feature13_N <dbl>,
## #   Feature14_N <dbl>, Feature15_N <dbl>, Feature16_N <dbl>, Feature17_N <dbl>,
## #   Feature18_N <dbl>, Feature19_N <dbl>, Feature20_N <dbl>, Feature21_N <dbl>,
## #   Feature22_N <dbl>, Feature23_N <dbl>, Feature24_N <dbl>, Feature25_N <dbl>,
## #   Feature26_N <dbl>, Feature27_N <dbl>, Feature28_N <dbl>, ...
```

Now we will move to the first screening method using Elastic Net (this will shrink collinear and unimportant chemicals coefficient to 0). (Elastic Net has L1 and L2 Regularization terms) Elastic net's penalty adds the following to the usual loss function of linear regression:

$$g(\beta) = \lambda[\alpha\|\beta\|_1 + \frac{1-\alpha}{2}\|\beta\|_2^2]$$

```
set.seed(123)
```

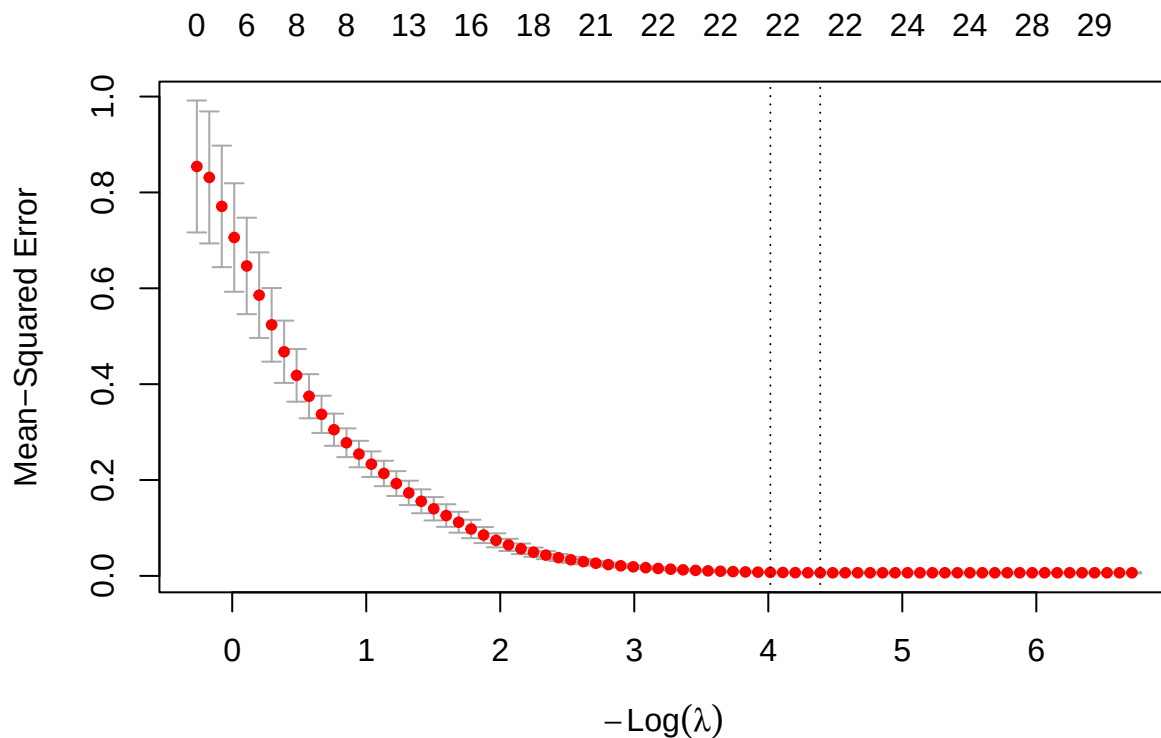
```
X <- screen_df %>%
  select(all_of(chem_cols)) %>%
  as.matrix()
```

```

y <- screen_df[["mean_resid_subject"]]

cv_enet <- cv.glmnet(
  x = X,
  y = y,
  alpha = 0.5,           # elastic net
  family = "gaussian",
  standardize = TRUE,
  nfolds = min(10, nrow(X))
)
plot(cv_enet)

```



lambda is the regularization term, picking between $-\log(\lambda)$ between 2.5 and 3 seems reasonable, bigger lambda's seem to have diminishing returns and provide a more complex model. Let's consider $-\log(\lambda) = 2.8$, i.e. $\lambda = \exp(-2.8)$, we will compare this to the optimal lambda from cross validation and check which gives us better results, I'd like a simpler model.

```

lambda_custom <- exp(-2.8)
lambda_custom

```

```
## [1] 0.06081006
```

Comparing Elastic Net models:

```

coef_min <- coef(cv_enet, s = "lambda.min")
coef_1se <- coef(cv_enet, s = "lambda.1se")
coef_custom <- coef(cv_enet, s = lambda_custom)

```

```

coef_min_df <- tibble(
  chemical = rownames(coef_min),
  coef_lambda_min = as.numeric(coef_min)
) %>%
  filter(chemical != "(Intercept)") %>%
  mutate(
    selected_lambda_min = coef_lambda_min != 0,
    abs_coef_lambda_min = abs(coef_lambda_min)
  )

coef_1se_df <- tibble(
  chemical = rownames(coef_1se),
  coef_lambda_1se = as.numeric(coef_1se)
) %>%
  filter(chemical != "(Intercept)") %>%
  mutate(
    selected_lambda_1se = coef_lambda_1se != 0,
    abs_coef_lambda_1se = abs(coef_lambda_1se)
  )

coef_custom_df <- tibble(
  chemical = rownames(coef_custom),
  coef_custom = as.numeric(coef_custom)
) %>%
  filter(chemical != "(Intercept)") %>%
  mutate(
    selected_custom = coef_custom != 0,
    abs_coef_custom = abs(coef_custom)
  )

enet_compare <- coef_min_df %>%
  select(chemical, coef_lambda_min, selected_lambda_min,
    abs_coef_lambda_min) %>%
  left_join(
    coef_1se_df %>%
      select(chemical, coef_lambda_1se,
        selected_lambda_1se, abs_coef_lambda_1se),
    by = "chemical"
  ) %>%
  left_join(
    coef_custom_df %>%
      select(chemical, coef_custom, selected_custom, abs_coef_custom),
    by = "chemical"
  ) %>%
  arrange(desc(abs_coef_lambda_min))

#enet_compare

shortlist_min <- enet_compare %>%
  filter(selected_lambda_min) %>%
  arrange(desc(abs_coef_lambda_min))

shortlist_1se <- enet_compare %>%
  filter(selected_lambda_1se) %>%

```

```

  arrange(desc(abs_coef_lambda_1se))

shortlist_custom <- enet_compare %>%
  filter(selected_custom) %>%
  arrange(desc(abs_coef_custom))

write.csv(enet_compare, "Elastic_Net_Chemical_Screening_Results.csv",
  row.names = F)

```

Classical SIS screening (using pairwise Pearson correlation of X v/s y to screen):

```

library(dccpp)
library(tidyverse)
sis_linear_tbl <- tibble(
  chemical = chem_cols,
  sis_corr = map_dbl(chem_cols, ~ cor(X[, .x], y, use = "complete.obs")),
  sis_abs_corr = abs(sis_corr)
) %>%
  arrange(desc(sis_abs_corr)) %>%
  mutate(
    sis_rank = row_number(),
    sis_top15 = sis_rank <= 15,
    sis_top20 = sis_rank <= 20,
    sis_top25 = sis_rank <= 25
  )

sis_dcorr_tbl <- tibble(
  chemical = chem_cols,
  dc_sis = map_dbl(chem_cols, ~ dccpp::dcor(X[, .x], y))
) %>%
  arrange(desc(dc_sis)) %>%
  mutate(
    dc_sis_rank = row_number(),
    dc_sis_top15 = dc_sis_rank <= 15,
    dc_sis_top20 = dc_sis_rank <= 20,
    dc_sis_top25 = dc_sis_rank <= 25
  )

```

Linear SIS uses pearson correlation so we are considering linear relationships, the other SIS considers distance correlation and is basically looking at association, so it captures non-linear relationships.

Compiled Table with SIS results

```

sis_compare <- sis_linear_tbl %>%
  select(
    chemical,
    sis_corr,
    sis_abs_corr,
    sis_rank,
    sis_top15,
    sis_top20,
    sis_top25
  ) %>%
  left_join(
    sis_dcorr_tbl %>%
      select(

```



```

    chemical,
    dc_sis,
    dc_sis_rank,
    dc_sis_top15,
    dc_sis_top20,
    dc_sis_top25
  ),
  by = "chemical"
) %>%
mutate(
  mean_sis_rank = (sis_rank + dc_sis_rank) / 2,
  selected_by_both_top15 = sis_top15 & dc_sis_top15,
  selected_by_both_top20 = sis_top20 & dc_sis_top20,
  selected_by_both_top25 = sis_top25 & dc_sis_top25,
  selected_by_either_top15 = sis_top15 | dc_sis_top15,
  selected_by_either_top20 = sis_top20 | dc_sis_top20,
  selected_by_either_top25 = sis_top25 | dc_sis_top25
) %>%
  arrange(mean_sis_rank)

#head(sis_compare)

write.csv(sis_compare, "SIS_Chemical_Screening_Results.csv", row.names = FALSE)

```

PCA finds directions explaining variation in X. PLS finds directions in X that are most related to Y. Sparse PLS forces many variable weights to 0 so only selected predictors define the components. Other than that, it works similar to PLS.

SPLS: Tuning hyperparameters for SPLS:

Run this chunk to install mixOmics, you might need to do something different for a different version of R:

```

#install.packages("BiocManager")
#options(repos = BiocManager::repositories())
#getOption("repos") #- make sure this shows you all repos not just CRAN
#install.packages("ggrepel", repos = "https://cloud.r-project.org", type = "binary")
#BiocManager::install("mixOmics", ask = FALSE, update = FALSE)
#library(mixOmics)

```

```

# options(repos = c(CRAN = "https://cloud.r-project.org"))
# install.packages("ggrepel")
# R.version.string
# sessionInfo()

```

```

#install.packages("BiocManager")
#BiocManager::install("mixOmics")
library(mixOmics)

```

```
## Loading required package: MASS
```

```
##
```

```
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
##      select
```

```
## Loading required package: lattice

##
## Loaded mixOmics 6.34.0
## Thank you for using mixOmics!
## Tutorials: http://mixomics.org
## Bookdown vignette: https://mixomicsteam.github.io/Bookdown
## Questions, issues: Follow the prompts at http://mixomics.org/contact-us
## Cite us: citation('mixOmics')

##
## Attaching package: 'mixOmics'

## The following object is masked from 'package:purrr':
##
##      map

set.seed(42)
tune_spls_res <- tune.spls(
  X = as.matrix(X),
  Y = y,
  ncomp = 2,
  test.keepX = c(5, 10, 15, 20, 25),
  validation = "Mfold",
  folds = 5,
  nrepeat = 20,
  measure = "MSE",
  mode = "regression",
  progressBar = TRUE
)

##
## comp 1
##      |
```

```
best_keepX <- tune_spls_res$choice.keepX
best_ncomp <- tune_spls_res$choice.ncomp

paste0("Optimal keepX: ", best_keepX, ", optimal ncomp: ", best_ncomp$ncomp)
```

```
## [1] "Optimal keepX: 25, optimal ncomp: 2" "Optimal keepX: 5, optimal ncomp: 2"
```

```
KeepX = (15, 20), 20, ncomp = 2
```

2 PCs will be retained and 15 variables load on one PC and 20 on the other. Variables are chosen through latent component optimization criterion, which is based on constructing components that maximize covariance with the response while imposing sparsity. There could be cross loadings.

I'd be inclined to think about the PCs as latent factors and less like just some projections.

Fitting the tuned models:

```
best_keepX <- c(15, 20)
best_ncomp <- 2
X_mat <- as.matrix(X)
spls_fit <- spls(
```

```

X = X_mat,
Y = y,
ncomp = best_ncomp,
keepX = best_keepX,
mode = "regression"
)

#Extracting selected variables
loadings_mat <- spls_fit$loadings$X

selected_list <- lapply(seq_len(best_ncomp), function(j) {
  rownames(mixOmics::selectVar(spls_fit, comp = j)$X$value)
})

names(selected_list) <- paste0("comp", seq_len(best_ncomp))

all_selected <- unique(unlist(selected_list))

spls_tbl <- tibble(
  chemical = rownames(loadings_mat),
  spls_selected = chemical %in% all_selected
)

```

Creating the results table:

```

for (j in seq_len(ncol(loadings_mat))) {
  spls_tbl[[paste0("loading_comp", j)]] <- as.numeric(loadings_mat[, j])
}

loading_cols <- grep("^loading_comp", names(spls_tbl), value = TRUE)

spls_tbl <- spls_tbl %>%
  mutate(
    spls_importance = rowSums(across(all_of(loading_cols), ~ abs(.x)))
  ) %>%
  arrange(desc(spls_importance)) %>%
  mutate(
    spls_rank = row_number(),
    spls_top15 = spls_rank <= 15,
    spls_top20 = spls_rank <= 20,
    spls_top25 = spls_rank <= 25
  )

for (j in seq_len(best_ncomp)) {
  spls_tbl[[paste0("selected_comp", j)]] <- spls_tbl$chemical %in% selected_list[[j]]
}

```

Saving the results:

```
write.csv(spls_tbl, "Sparse_PLS_Chemical_Screening_Results.csv", row.names = F)
```

Finally, we'll screen variables and give ranks based on XGBoost (ML model)

This is probably too complex of a solution but I hope when we are considering an ensemble of the different screening techniques I'll give low weights to XGBoost rankings so that this does not sway our final ranking

too much.

```
library(xgboost)
set.seed(42)
dtrain <- xgb.DMatrix(data = X_mat, label = y)

xgb_fit <- xgb.train(
  params = list(
    objective = "reg:squarederror",
    eval_metric = "rmse",
    eta = 0.05,
    max_depth = 4,
    min_child_weight = 3,
    subsample = 0.8,
    colsample_bytree = 0.8,
    lambda = 1,
    alpha = 0
  ),
  data = dtrain,
  nrounds = 200,
  verbose = 0
)

xgb_imp <- xgb.importance(
  feature_names = colnames(X),
  model = xgb_fit
)
```

xgb_gain is the entropy gain or how important the variable is. xgb_cover shows how broadly the variable's splits affect observations. xgb_frequency is how often the variable was used in splitting

Use the Gain to decide importance.

Creating a results table for XGBoost:

```
xgb_tbl <- tibble(chemical = colnames(X)) %>%
  left_join(
    as_tibble(xgb_imp) %>%
      transmute(
        chemical = Feature,
        xgb_gain = Gain,
        xgb_cover = Cover,
        xgb_frequency = Frequency
      ),
    by = "chemical"
  ) %>%
  mutate(
    xgb_gain = replace_na(xgb_gain, 0),
    xgb_cover = replace_na(xgb_cover, 0),
    xgb_frequency = replace_na(xgb_frequency, 0)
  ) %>%
  arrange(desc(xgb_gain)) %>%
  mutate(
    xgb_rank = row_number(),
    xgb_top15 = xgb_rank <= 15,
    xgb_top20 = xgb_rank <= 20,
```

```

    xgb_top25 = xgb_rank <= 25
  )
write.csv(xgb_tbl, "XGBoost_Screening_Results.csv", row.names = FALSE)

```

Giving ranks to Elastic Net Screening:

```

enet_compare <- enet_compare %>%
  mutate(
    enet_rank_lambda_min = min_rank(desc(abs_coef_lambda_min)),
    enet_rank_lambda_1se = min_rank(desc(abs_coef_lambda_1se)),
    enet_rank_custom = min_rank(desc(abs_coef_custom)),

    enet_top15_lambda_min = enet_rank_lambda_min <= 15,
    enet_top20_lambda_min = enet_rank_lambda_min <= 20,
    enet_top25_lambda_min = enet_rank_lambda_min <= 25,

    enet_top15_lambda_1se = enet_rank_lambda_1se <= 15,
    enet_top20_lambda_1se = enet_rank_lambda_1se <= 20,
    enet_top25_lambda_1se = enet_rank_lambda_1se <= 25,

    enet_top15_custom = enet_rank_custom <= 15,
    enet_top20_custom = enet_rank_custom <= 20,
    enet_top25_custom = enet_rank_custom <= 25
  )

enet_compare <- enet_compare %>%
  mutate(
    enet_rank_lambda_min = rank(-abs_coef_lambda_min, ties.method = "min"),
    enet_rank_lambda_1se = rank(-abs_coef_lambda_1se, ties.method = "min"),
    enet_rank_custom = rank(-abs_coef_custom, ties.method = "min")
  )

head(enet_compare)

```

```

## # A tibble: 6 x 22
##   chemical      coef_lambda_min selected_lambda_min abs_coef_lambda_min
##   <chr>          <dbl> <lgl>                <dbl>
## 1 Feature34_N      0.331 TRUE                0.331
## 2 Feature9_N      -0.328 TRUE                0.328
## 3 Feature30_N      0.246 TRUE                0.246
## 4 Feature1_N       0.164 TRUE                0.164
## 5 Feature5_N      -0.145 TRUE                0.145
## 6 Feature40_N     -0.114 TRUE                0.114
## # i 18 more variables: coef_lambda_1se <dbl>, selected_lambda_1se <lgl>,
## #   abs_coef_lambda_1se <dbl>, coef_custom <dbl>, selected_custom <lgl>,
## #   abs_coef_custom <dbl>, enet_rank_lambda_min <int>,
## #   enet_rank_lambda_1se <int>, enet_rank_custom <int>,
## #   enet_top15_lambda_min <lgl>, enet_top20_lambda_min <lgl>,
## #   enet_top25_lambda_min <lgl>, enet_top15_lambda_1se <lgl>,
## #   enet_top20_lambda_1se <lgl>, enet_top25_lambda_1se <lgl>, ...

```

Collating all Screening methods result to get one table:

```

all_screening_tbl <- enet_compare %>%

```

```

left_join(
  sis_compare,
  by = "chemical"
) %>%
left_join(
  spls_tbl,
  by = "chemical"
) %>%
left_join(
  xgb_tbl,
  by = "chemical"
)

head(all_screening_tbl)

## # A tibble: 6 x 57
##   chemical      coef_lambda_min selected_lambda_min abs_coef_lambda_min
##   <chr>          <dbl> <lgl>              <dbl>
## 1 Feature34_N      0.331 TRUE              0.331
## 2 Feature9_N       -0.328 TRUE              0.328
## 3 Feature30_N      0.246 TRUE              0.246
## 4 Feature1_N       0.164 TRUE              0.164
## 5 Feature5_N      -0.145 TRUE              0.145
## 6 Feature40_N     -0.114 TRUE              0.114
## # i 53 more variables: coef_lambda_1se <dbl>, selected_lambda_1se <lgl>,
## #   abs_coef_lambda_1se <dbl>, coef_custom <dbl>, selected_custom <lgl>,
## #   abs_coef_custom <dbl>, enet_rank_lambda_min <int>,
## #   enet_rank_lambda_1se <int>, enet_rank_custom <int>,
## #   enet_top15_lambda_min <lgl>, enet_top20_lambda_min <lgl>,
## #   enet_top25_lambda_min <lgl>, enet_top15_lambda_1se <lgl>,
## #   enet_top20_lambda_1se <lgl>, enet_top25_lambda_1se <lgl>, ...

#Adding some summaries which will help me decide weights and validate assumptions
all_screening_tbl <- all_screening_tbl %>%
  mutate(
    mean_rank_all = rowMeans(
      cbind(
        sis_rank,
        dc_sis_rank,
        enet_rank_lambda_min,
        enet_rank_lambda_1se,
        enet_rank_custom,
        spls_rank,
        xgb_rank
      )
    ),
    na.rm = TRUE
  ),

  n_top15_methods =
    sis_top15 + dc_sis_top15 +
    enet_top15_lambda_min + enet_top15_lambda_1se + enet_top15_custom +
    spls_top15 + xgb_top15,

  n_top20_methods =

```

```

    sis_top20 + dc_sis_top20 +
    enet_top20_lambda_min + enet_top20_lambda_1se + enet_top20_custom +
    spls_top20 + xgb_top20,

    n_top25_methods =
    sis_top25 + dc_sis_top25 +
    enet_top25_lambda_min + enet_top25_lambda_1se + enet_top25_custom +
    spls_top25 + xgb_top25
  ) %>%
  arrange(desc(n_top15_methods), mean_rank_all)

write.csv(all_screening_tbl, "Collated_Screening_Results.csv", row.names = FALSE)

```

Data Visualisation for comparing different methods:

```

library(tidyverse)
library(ggplot2)
library(forcats)

heatmap_tbl <- all_screening_tbl %>%
  slice_min(mean_rank_all, n = 20) %>%
  dplyr::select(
    chemical,
    sis_rank,
    dc_sis_rank,
    enet_rank_lambda_min,
    enet_rank_lambda_1se,
    enet_rank_custom,
    spls_rank,
    xgb_rank
  ) %>%
  pivot_longer(
    cols = -chemical,
    names_to = "method",
    values_to = "rank"
  ) %>%
  mutate(
    method = factor(
      method,
      levels = c(
        "sis_rank",
        "dc_sis_rank",
        "enet_rank_lambda_min",
        "enet_rank_lambda_1se",
        "enet_rank_custom",
        "spls_rank",
        "xgb_rank"
      )
    ),
    labels = c(
      "SIS",
      "DC-SIS",
      "ENet min",
      "ENet 1se",

```

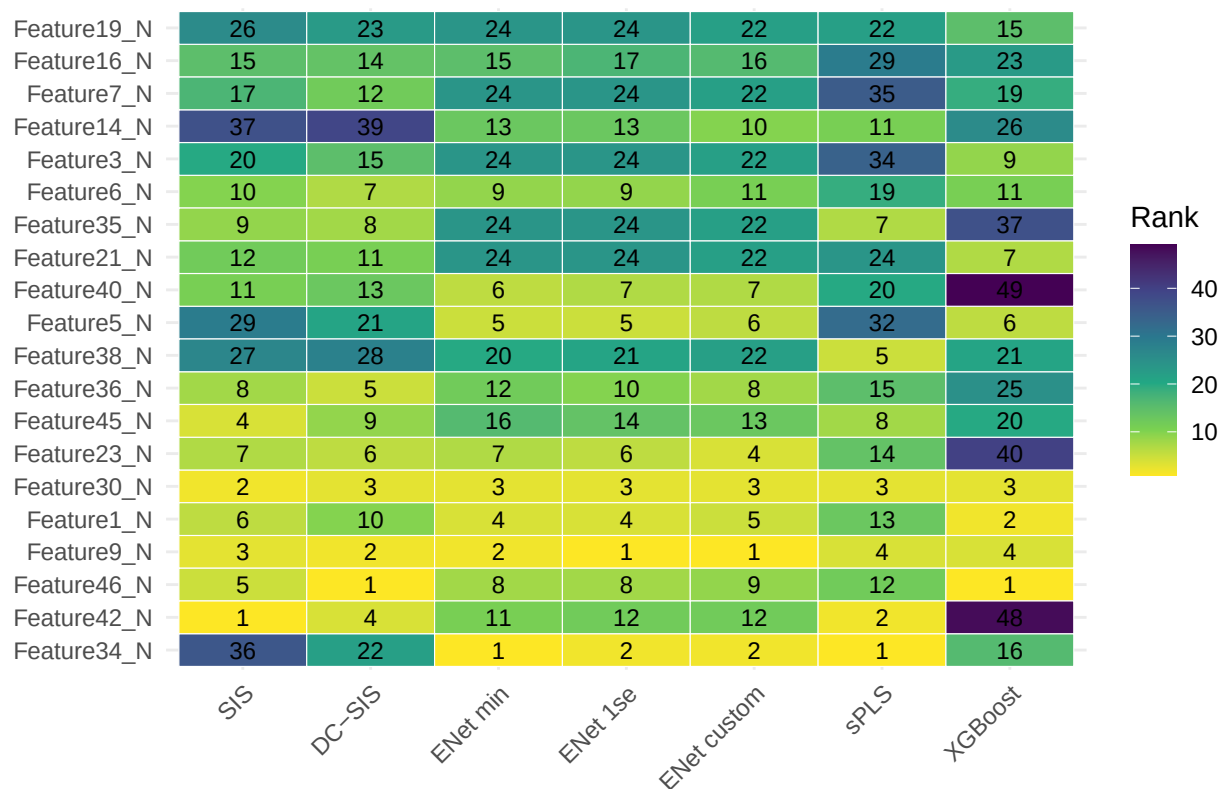
```

      "ENet custom",
      "sPLS",
      "XGBoost"
    )
  )
)

ggplot(heatmap_tbl,
  aes(x = method,
      y = fct_reorder(chemical, rank, .fun = min),
      fill = rank)) +
  geom_tile(color = "white") +
  geom_text(aes(label = rank), size = 3) +
  scale_fill_viridis_c(direction = -1) +
  labs(
    x = NULL,
    y = NULL,
    fill = "Rank",
    title = "Top 20 chemicals across screening methods"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1)
  )

```

Top 20 chemicals across screening methods



Bump Chart:


```

library(tidyverse)
library(ggplot2)

bump_tbl <- all_screening_tbl %>%
  slice_min(mean_rank_all, n = 10) %>%
  dplyr::select(
    chemical,
    sis_rank,
    dc_sis_rank,
    enet_rank_lambda_min,
    enet_rank_lambda_1se,
    enet_rank_custom,
    spls_rank,
    xgb_rank
  ) %>%
  pivot_longer(
    cols = -chemical,
    names_to = "method",
    values_to = "rank"
  ) %>%
  mutate(
    method = factor(
      method,
      levels = c(
        "sis_rank",
        "dc_sis_rank",
        "enet_rank_lambda_min",
        "enet_rank_lambda_1se",
        "enet_rank_custom",
        "spls_rank",
        "xgb_rank"
      ),
      labels = c(
        "SIS",
        "DC-SIS",
        "ENet min",
        "ENet 1se",
        "ENet custom",
        "sPLS",
        "XGBoost"
      )
    )
  )

ggplot(bump_tbl,
  aes(x = method, y = rank, group = chemical, color = chemical)) +
  geom_line(size = 1.1, smooth = 8) +
  geom_point(size = 2) +
  scale_y_reverse(breaks = 1:20) +
  labs(
    x = NULL,
    y = "Rank",
    title = "Rank movement across screening methods"
  )

```

```

) +
theme_minimal() +
theme(
  axis.text.x = element_text(angle = 45, hjust = 1)
)

```

```

## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

## Warning in geom_line(size = 1.1, smooth = 8): Ignoring unknown parameters:
## `smooth`

```

